GATE CS — Digital Logic

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
Boolean algebra & K-map	2.5	Tier 2
Number representation, IEEE-754, arithmetic	2.3	Tier 2
Sequential circuits (flip-flops, counters, FSM)	1.2	Tier 3
Combinational circuits (MUX/decoder)	0.9	Tier 3

Digital Logic carries about **6.9 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Digital_Logic.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Boolean Algebra and K-Maps

- Laws: identity (A + 0 = A, A·1 = A), null (A + 1 = 1, A·0 = 0), idempotent, complement (A + A′ = 1, A·A′ = 0), involution, commutative, associative, distributive (A + BC = (A + B)(A + C)), **De Morgan** ((A + B)′ = A′B′, (AB)′ = A′ + B′), absorption (A + AB = A), consensus (AB + A′C + BC = AB + A′C), A + A′B = A + B.
- **Duality:** swap AND/OR and 0/1 to get the dual identity. Number of Boolean functions of n variables: 2^(2ⁿ) (n = 2 → 16, n = 3 → 256).
- **Canonical forms:** sum of minterms Σm (each minterm is a product of all variables; F = 1) and product of maxterms ΠM (each maxterm is a sum of all variables; F = 0). Minterm m_i and maxterm M_i are complements. Number of minterms = 2^n. Σm(1, 2, 4, 7) of three variables is the XOR/odd-parity function A ⊕ B ⊕ C.
- **XOR properties:** A ⊕ B = A′B + AB′; A ⊕ A = 0; A ⊕ 0 = A; A ⊕ 1 = A′; commutative and associative. XNOR (equivalence) = (A ⊕ B)′; for n variables, XOR is 1 for an odd number of ones.
- **Universal gates:** NAND and NOR. Making NOT, AND, OR from NAND: NOT = NAND with tied inputs; AND = NAND followed by NOT; OR = NAND of the complemented inputs. Total NAND gates for XOR: 4. Self-dual functions have $f(x_1 \ldots x_n) = f'(x_1' \ldots x_n')$.

Karnaugh maps

- Gray-code ordering along each axis so neighbours differ in one variable. Group 1s (for SOP) in rectangles of size 1, 2, 4, 8… (powers of two); the map wraps around the edges. Bigger groups → fewer literals: a group of 2^k cells eliminates k variables.
- **Prime implicant**: a maximal group. **Essential prime implicant**: covers a minterm no other prime implicant covers. Minimal SOP = all essential prime implicants + enough others to cover the rest.
- **Don't cares (X):** include them in groups only if they make a group larger. For POS, group the 0s and complement the result.
- Number of gates/literals for a minimal expression: count after minimisation. Static hazards appear when adjacent groups are not overlapped.

2. Number Representation, IEEE-754, Arithmetic

- **Radix conversion:** binary ↔ octal (groups of 3 bits), binary ↔ hex (groups of 4). Fractions: multiply by the base repeatedly. n bits represent 2^n values.
- **Signed integers (n bits):**

Scheme	Range	Notes
Sign-magnitude	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	two zeros
1's complement	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	two zeros; negate by inverting
2's complement	-2^{n-1} … $+(2^{n-1}-1)$	one zero; negate = invert + 1
Excess-K (biased)	$-K$ … 2^n-1-K	used for exponents

- 2's complement: value = $-b_{n-1}\cdot 2^{n-1} + \Sigma\, b_i 2^i$. Sign extension copies the sign bit. 10110110 (8 bits) = −74. Adding numbers: ignore the carry out; **overflow** occurs when two operands of the same sign give a result of the opposite sign (equivalently, carry into the sign bit ≠ carry out of it). Unsigned overflow = carry out of the MSB.
- Subtraction A − B = A + (2's complement of B). Multiplication by 2^k = left shift; division by 2^k = arithmetic right shift (sign preserved). **Booth's algorithm** multiplies signed numbers using recoding of runs of 1s.
- BCD: 4 bits per decimal digit; add 6 when a digit sum exceeds 9. Gray code: successive values differ in one bit; binary → Gray: g = b ⊕ (b ≫ 1). Parity bits and Hamming codes for error detection.
- **IEEE-754 single precision (32 bits):** 1 sign bit, 8 exponent bits (bias 127), 23 fraction bits; value = (−1)^s × 1.f × 2^(e − 127) for normalised numbers. **Double (64 bits):** 1 + 11 (bias 1023) + 52. Exponent field all 0s → zero/denormal (no hidden 1); all 1s → infinity (fraction 0) or NaN. Example: 0xC1A40000 → sign 1, exponent 10000011 = 131 → 2^4, fraction .0100100 → 1.28125 → −20.5.
- Converting a decimal to IEEE-754: write in binary, normalise to 1.xxx × 2^e, add the bias to e, fill the fraction. Floating-point addition: align exponents, add mantissas, normalise, round. Precision is finite: 0.1 cannot be represented exactly. Largest single-precision finite number ≈ 3.4×10^{38}.

Arithmetic circuits

- **Half adder:** Sum = A ⊕ B, Carry = AB. **Full adder:** Sum = A ⊕ B ⊕ Cin, Cout = AB + Cin(A ⊕ B). Full adder = 2 half adders + OR.
- **Ripple-carry adder:** n full adders, delay grows with n (carry ripples). **Carry look-ahead:** generate $G_i = A_iB_i$ and propagate $P_i = A_i \oplus B_i$; $C_{i+1} = G_i + P_iC_i$; computes all carries in parallel (constant gate delays per group).
- Adder/subtractor: XOR the B inputs with a control bit M and feed M as the carry-in. Overflow detection from the last two carries. Comparator, ALU design, array and Wallace-tree multipliers.

3. Combinational Circuits (MUX, Decoder, Encoder)

- Output depends only on current inputs (no memory). Analysis: write each gate's expression, then the truth table.
- **Multiplexer (2^n:1):** n select lines choose one of 2^n data inputs. Output = Σ (select minterm × data input). Any function of n + 1 variables can be built with a 2^n:1 MUX by using n variables as selects and feeding each data input with 0, 1, the remaining variable or its complement. Example: F(A,B,C) = Σm(1,2,4,7) with A, B as selects: $I_0 = C$, $I_1 = C'$, $I_2 = C'$, $I_3 = C$. A 2^n:1 MUX can implement any n-variable function with constant data inputs. A 2^n:1 MUX from 2:1 MUXes needs $2^n - 1$ of them.
- **Demultiplexer:** one input routed to one of 2^n outputs. **Decoder (n → 2^n):** exactly one output high for each input combination; OR the required minterm outputs to realise any function (n-input decoder + OR gates, or NAND of active-low outputs). A 3-to-8 decoder + enable can be combined to make bigger decoders (a 4-to-16 decoder from two 3-to-8 decoders).
- **Encoder (2^n → n)** and **priority encoder** (highest-priority active input wins). Comparator, parity generator (XOR tree), code converters (BCD to seven segments).
- **ROM / PLA / PAL:** ROM = decoder + OR array (a $2^n \times m$ ROM implements m functions of n variables); PLA has programmable AND and OR planes. Look-up tables in FPGAs.
- **Hazards:** static hazards (glitches) from unequal gate delays; cured by adding redundant prime implicants. Propagation delay of a circuit = delay along the longest path.

4. Sequential Circuits

- Output depends on the inputs *and* stored state. **Latch** (level-sensitive) vs **flip-flop** (edge-triggered). All flip-flops change on a clock edge in synchronous circuits.
- **Flip-flop characteristic equations:** SR: $Q^+ = S + R'Q$ (with $S \cdot R = 0$); **D: $Q^+ = D$**; **JK: $Q^+ = JQ' + K'Q$** (J = K = 1 toggles; J = K = 0 holds); **T: $Q^+ = T \oplus Q$**. Convert JK to T by tying J = K = T; JK to D with J = D, K = D′. Master–slave configuration avoids the race-around condition in JK flip-flops.
- **Excitation tables:** to get $Q \to Q^+$: D = Q^+; T = $Q \oplus Q^+$; JK: 0 → 0 (J=0, K=X), 0 → 1 (J=1, K=X), 1 → 0 (J=X, K=1), 1 → 1 (J=X, K=0).
- **Counters:** n flip-flops count up to 2^n states (mod-2^n). Ripple (asynchronous) counter: each output toggles the next; the n-th output frequency = $f_{clk}/2^n$; delay accumulates. Synchronous counter: all flip-flops share the clock. A mod-N counter needs $\lceil \log_2 N \rceil$ flip-flops and a reset/feedback to skip states. **Ring counter:** n flip-flops give n states; **Johnson (twisted-ring) counter:** 2n states. Down counters, up/down counters, BCD (decade) counters.
- **Registers:** n flip-flops store n bits. **Shift registers:** SISO, SIPO, PISO, PIPO; shifting left by one position multiplies by 2; serial addition uses a shift register and one full adder.
- **State machines: Moore** (output depends only on the state) and **Mealy** (output depends on state and input). Analyse by writing the state table, then state diagram; sequence detectors (overlapping/non-overlapping) for patterns like 101. Minimum number of flip-flops = $\lceil \log_2$ (number of states)$\rceil$. To find the state after k clock pulses, compute the cycle and take k mod (cycle length).
- **Timing:** setup time (data stable before the clock edge), hold time (stable after), clock-to-Q delay. Maximum clock frequency: $T_{clk} \geq t_{clk\text{-}to\text{-}Q} + t_{logic} + t_{setup}$ (+ skew). Clock skew and metastability.
- Memory cells: SRAM (6 transistors), DRAM (1 transistor + capacitor, refresh needed).

5. Exam strategy & revision checklist

- Study order: K-map minimisation and minterm/maxterm manipulation → number systems and IEEE-754 → counters/flip-flop analysis → MUX/decoder implementations.
- Questions are short: write the truth table first. Verify a K-map answer by checking two or three minterms.
- ☐ Minimise with K-maps (with don't cares). ☐ 2's complement arithmetic and overflow. ☐ IEEE-754 encode/decode. ☐ Implement a function with a MUX and a decoder. ☐ Flip-flop equations and conversions. ☐ State sequence of a counter after k pulses. ☐ Redo `GATE_CS_Digital_Logic.pdf` past questions.